

REQUIREMENTS-DRIVEN SEMANTIC GATEWAY ARCHITECTURE

*Integrating JTBD · Persona Needs · Functional & Non-Functional Requirements
into the Ontology, SHACL/SWRL Rules Store & Domain Vector Databases*

Extension to: Semantic Gateway Architecture v1



1. The Problem: Requirements as First-Class Semantic Citizens

Most enterprise AI deployments treat requirements as upstream artifacts that live in product management tools and get forgotten the moment a system is built. The result: the AI operates without knowing why it exists, who it serves, what job it is trying to accomplish, or what constitutes success for that job. When the AI fails, there is no requirements baseline to trace the failure back to.

This extension to the Semantic Gateway Architecture solves that by treating requirements as ontology-defined entities with formal structure, machine-readable semantics, and direct linkage to the SHACL constraints, SWRL rules, and vector database indices that govern AI behavior. Requirements do not sit in a separate system — they become the semantic scaffolding through which every other architectural element is organized and validated.

Governing Principle

A requirement is not a document. It is an ontology individual of class Requirement, with typed properties for persona, job, outcome, acceptance criterion, domain, and constraint linkage. It lives in the triplestore, is enforced by SHACL shapes, activates SWRL rules, and seeds the domain vector databases. Every AI action is traceable to one or more governing requirements.

2. The Requirements Ontology

The Requirements Ontology is a new domain module in the federated ontology stack, positioned above the upper ontology and below all domain-specific ontologies. It provides the shared vocabulary for expressing what people need, why they need it, and what good looks like — in a form the Semantic Gateway can reason over.

2.1 Core Class Hierarchy

Ontology Class	Superclass	What It Represents
Requirement	owl:Thing	The root class for all requirements — any stated need that must be satisfied by the system
PersonaRequirement	Requirement	A need tied to a specific Persona, derived from their role, responsibilities, and domain context
JobRequirement	PersonaRequirement	A need derived from a specific Job-to-Be-Done that the persona is trying to accomplish
OutcomeRequirement	JobRequirement	The desired end-state the persona wants to reach by completing the job — the 'so that' clause
FunctionalRequirement	Requirement	A specific capability the system must provide (what it must do)
NonFunctionalRequirement	Requirement	A quality attribute the system must exhibit (how it must do it) — performance, reliability, usability

ComplianceRequirement	Requirement	A requirement derived from a regulatory obligation, standard, or policy — what the system must enforce
ControlRequirement	ComplianceRequirement	A specific control mechanism that must be implemented — often 1:1 with a SHACL shape or SWRL rule
ProcessRequirement	Requirement	A process step or workflow pattern that must be supported within a domain or sub-domain
ScenarioRequirement	Requirement	A scenario-specific instantiation — captures the context, preconditions, and expected behavior for a particular use case
AcceptanceCriterion	owl:Thing	A testable condition that, when met, proves a requirement is satisfied

2.2 Core Properties

Property	Domain	Range	Meaning
req:forPersona	Requirement	Persona	The persona whose need this requirement expresses
req:describesJob	JobRequirement	Job	The JTBD this requirement is derived from
req:targetsOutcome	OutcomeRequirement	DesiredOutcome	The outcome the persona wants to achieve
req:inDomain	Requirement	BusinessDomain	The business domain this requirement belongs to
req:inSubDomain	Requirement	BusinessSubDomain	The sub-domain (e.g., Credit Risk within Risk Management)
req:governedBy	Requirement	ComplianceObligation	The regulation, standard, or policy that creates this requirement
req:enforcedBy	ControlRequirement	SHACLShape SWRLRule	The SHACL shape or SWRL rule that implements this control
req:hasCriterion	Requirement	AcceptanceCriterion	One or more testable acceptance criteria
req:priority	Requirement	xsd:string	MoSCoW classification: Must Should Could WontHave
req:status	Requirement	xsd:string	Lifecycle state: Draft Validated Active Deprecated
req:scenarioContext	ScenarioRequirement	ScenarioContext	The specific situational context that activates this requirement
req:dependsOn	Requirement	Requirement	Prerequisite requirements that must be satisfied first
req:conflictsWith	Requirement	Requirement	Requirements that cannot both be satisfied simultaneously
req:version	Requirement	xsd:string	Semantic version of this requirement (e.g., 2.1.0)

req:rationale	Requirement	xsd:string	The business reason or evidence base for this requirement
---------------	-------------	------------	---

3. Jobs-to-Be-Done as Semantic Anchors

The Jobs-to-Be-Done framework provides the motivational layer beneath requirements. A Job is not a task — it is the progress a persona is trying to make in a given context. Jobs are stable over time even when the features that serve them change. By anchoring requirements to Jobs, the architecture gains an interpretive layer that the Semantic Gateway uses to validate whether a prompt is aligned with what the persona actually needs to accomplish.

3.1 The Job Ontology Module

Class / Property	Type	Description
Job	Class	A functional unit of progress a persona hires a product/system to make (e.g., 'Assess project risk')
FunctionalJob	Subclass of Job	The core task objective — what the persona is literally trying to do
EmotionalJob	Subclass of Job	How the persona wants to feel while doing it (e.g., confident, in control)
SocialJob	Subclass of Job	How the persona wants to be perceived by others while doing it
DesiredOutcome	Class	A metric the persona uses to measure success at the job — direction + metric + object + context
JobContext	Class	The situational circumstances that activate or shape the job (time pressure, data availability, stakeholder visibility)
JobBarrier	Class	A friction point or obstacle that prevents the persona from getting the job done
job:performedBy	Property	Links a Job to the Persona class(es) that perform it
job:measuredBy	Property	Links a Job to its DesiredOutcome instances
job:activatedIn	Property	Links a Job to its JobContext instances
job:blockedBy	Property	Links a Job to JobBarrier instances the gateway should help resolve
job:satisfiedBy	Property	Links a Job to FunctionalRequirement instances that address it

3.2 Illustrative JTBD Instantiation: Risk Manager Persona

The following shows how a single persona's job hierarchy is represented as ontology individuals. This instance graph is what the Semantic Gateway resolves when the Risk Manager submits a prompt:

JTBD Instance Graph — Risk Manager
Persona: RiskManager_Senior → FunctionalJob: 'Assess residual risk across active digital twin projects' → JobContext: 'Quarterly board reporting cycle, limited time, stakeholder visibility HIGH' → DesiredOutcome[1]: 'Minimize time to produce complete risk assessment from 3 days to 4 hours' → DesiredOutcome[2]: 'Increase confidence that all ISO 31000 criteria are covered from 70% to 100%' → JobBarrier[1]: 'Risk data scattered across 6 disparate systems' → JobBarrier[2]: 'Inconsistent risk terminology between project teams and corporate risk function' → JobRequirement: 'System must resolve risk entity terminology to corporate risk ontology before assessment' → ControlRequirement: 'Assessment output must be validated against ISO_31000_Shape before delivery'

4. Requirements Capture Pipeline

Requirements enter the architecture through a structured elicitation and formalization pipeline. The pipeline transforms qualitative stakeholder input into formally typed ontology individuals, validated against the Requirements Ontology, before they are stored in the triplestore and indexed into the domain vector databases.

4.1 Five-Stage Capture Pipeline

Stage 1: Elicitation Interface
A structured requirements capture form or conversational interface that prompts stakeholders using JTBD-derived question templates: 'Who are you?' (Persona), 'What are you trying to accomplish?' (Functional Job), 'What would success look like?' (Desired Outcome), 'What gets in your way?' (Job Barrier), 'What constraints must be respected?' (Compliance/Control). Outputs a draft Requirement Record.

Stage 2: Ontology Classification
The draft Requirement Record is passed to the Semantic Gateway for classification. The gateway maps the stated need to Requirement subclasses (Functional, NonFunctional, Compliance, Control, Process, Scenario), resolves the persona reference, links the job and desired outcome to existing Job ontology individuals or creates new ones, and assigns domain and sub-domain membership. Outputs a typed RequirementDraft individual.

Stage 3: SHACL Validation of the Requirement Itself
The RequirementDraft is validated against sh:RequirementCompleteness shape, which enforces: forPersona (cardinality 1), inDomain (cardinality 1), priority (enumerated MoSCoW), hasCriterion (minimum 1 AcceptanceCriterion), status (enumerated). Validation failures are returned to the elicitation interface with specific field guidance. Validated requirements receive status=Draft.

Stage 4: Stakeholder Review & Activation

Draft requirements enter a review workflow. Domain stewards validate domain/sub-domain assignment. Compliance officers validate ComplianceRequirement instances against applicable regulations. Conflicts and dependencies are resolved. Approved requirements transition to status=Active and are written as triples to the GraphDB triplestore with provenance metadata (who approved, when, ticket reference).

Stage 5: Vector DB Indexing

Active requirements are embedded and indexed into the domain vector database(s) matching their inDomain property. The embedding includes: the requirement text, persona context, job context, desired outcomes, acceptance criteria, and domain/sub-domain tags — enabling semantic retrieval by scenario similarity. Scenario-specific details are captured in ScenarioRequirement individuals and indexed with rich situational context metadata.

5. SHACL Shapes for Requirements — Relational Store

Following the same design pattern established in the core architecture, SHACL shapes governing requirements are stored as records in the relational database. This enables requirements governance teams to manage constraint evolution using familiar tooling without direct triplestore access.

5.1 Core Requirement Shapes

Shape Name	Target Class	Enforced Constraints
sh:RequirementCompleteness	Requirement	- req:forPersona exactly 1 - req:inDomain min 1 - req:priority in {Must, Should, Could, WontHave} - req:status in {Draft, Validated, Active, Deprecated} - req:hasCriterion min 1 - req:version matches semantic version pattern - req:rationale min 1 (length > 20 chars)
sh:JobRequirementLinkage	JobRequirement	- req:describesJob exactly 1 - req:targetsOutcome min 1 - job:measuredBy min 1 (DesiredOutcome with direction + metric) - job:activatedIn min 1 (JobContext)
sh:ComplianceRequirementShape	ComplianceRequirement	- req:governedBy min 1 (ComplianceObligation) - req:inDomain min 1 - Must link to applicable jurisdiction(s) - req:enforcedBy min 1 when status=Active
sh:ControlRequirementShape	ControlRequirement	- req:enforcedBy min 1 (SHACLShape OR SWRLRule) - Must have testability evidence (SPARQL test query or test fixture reference)

		Must have owner (responsible data steward or engineer)
sh:ScenarioRequirementShape	ScenarioRequirement	- req:scenarioContext exactly 1 - Precondition assertions min 1 - Trigger event or condition min 1 - Expected behavior description min 1 - At least 1 AcceptanceCriterion with SPARQL-evaluable test
sh:NonFunctionalRequirementShape	NonFunctionalRequirement	- nfr:qualityDimension in {Performance, Reliability, Security, Usability, Maintainability, Scalability} - nfr:measurableThreshold min 1 (numeric with unit) - nfr:measurementMethod min 1 - nfr:monitoringFrequency min 1

6. SWRL Rules Governing Requirement-Driven Behavior

SWRL rules stored in the relational DB fire on requirement class membership and job context to modify gateway behavior, activate domain agents, enforce acceptance criteria, and escalate conflicts. These rules transform passive requirement records into active behavioral drivers.

6.1 Key Rule Patterns

Rule ID	Trigger Condition	Inferred Consequence	Architectural Effect
REQ-001	SPO persona_id matches Persona P AND P performs FunctionalJob J AND J has JobContext C active	Attach J's DesiredOutcome metrics as success criteria to CAD	Gateway injects outcome measurement instructions into AI Directive
REQ-002	SPO intent resolves to domain D AND D has Active ScenarioRequirement S with matching preconditions	S's AcceptanceCriteria become validation gates on agent response	Service Facade validates agent output against S before delivery
REQ-003	Request targets Entity E AND E is in Scope of ComplianceRequirement CR AND CR has ControlRequirement CC	CC's enforcing SHACLShape is added to active constraint set	Gateway retrieves and applies CC's shape during compilation
REQ-004	FunctionalRequirement FR has status=Active AND FR has no enforcedBy linkage AND FR priority=Must	Flag FR as GOVERNANCE_GAP and attach to CAD as warning	Response includes governance gap notification for steward review
REQ-005	Persona has JobBarrier B with barrier_type=TerminologyInconsistency AND B applies to domain D	Force ontology disambiguation on all D-domain terms in SPO	Gateway triggers disambiguation before proceeding to SHACL evaluation

REQ-006	ScenarioRequirement S has status=Active AND SPO context matches S's scenarioContext	Retrieve S's scenario-specific vector DB entries as priority context	Agent receives scenario-specific retrieval results ranked above general domain results
REQ-007	NonFunctionalRequirement NFR applies to intent AND NFR quality_dimension=Performance AND NFR threshold=T	Attach performance SLA T to CAD execution parameters	Agent invocation is governed by T as timeout/resource ceiling
REQ-008	Two Active Requirements R1 and R2 have conflictsWith each other AND both are triggered by same SPO	Halt compilation, return REQUIREMENT_CONFLICT with R1 and R2 citations	Human resolution required before proceeding — conflict logged to governance queue

7. Gateway Integration: How Requirements Shape Prompt Compilation

When the Semantic Gateway receives a Structured Prompt Object, requirement resolution becomes Stage 0 — before ontology resolution, before SHACL evaluation, before PEP orchestration. Requirements define the lens through which everything else is interpreted.

7.1 Updated Gateway Compilation Pipeline

Stage 0 — Requirement Resolution (NEW)

The gateway queries the triplestore for all Active requirements matching: (a) the authenticated persona, (b) the resolved intent_tag mapped to a FunctionalJob class, and (c) the entity domain. The result is a Requirement Context Bundle (RCB) — the set of requirements that govern this specific interaction. The RCB travels with the SPO through every subsequent stage and is attached to the final CAD.

Stage 1 — Ontology Resolution (Enhanced)

Entity resolution now includes job context injection: resolved entities are annotated with the persona's active JobContext, which may modify property visibility (some properties are only relevant in specific job contexts), priority weighting (properties central to the job's DesiredOutcome are flagged as HIGH_RELEVANCE), and barrier context (if a JobBarrier relates to the entity type, disambiguation and quality checks are automatically elevated).

Stage 2 — SHACL Evaluation (Enhanced)

SHACL evaluation now includes requirement-derived shapes: ControlRequirement individuals with enforcedBy linkage inject their referenced shapes into the active evaluation set dynamically. The shape evaluation result is annotated with the requirement that activated it — creating a traceable chain from response constraint back to the original stakeholder need.

Stage 3 — SWRL Rule Execution (Enhanced)

Requirement-class SWRL rules (REQ-001 through REQ-008) fire in this stage. Their consequences modify the CAD: injecting DesiredOutcome metrics as success criteria, attaching AcceptanceCriteria as response validation gates, flagging governance gaps, and activating scenario-specific retrieval priorities.

Stage 4 — PEP Orchestration (Enhanced)

PEP evaluation is now requirement-aware. Each PEP receives the RCB and uses it to contextualize its verdict: the Compliance PEP knows which ComplianceRequirements are active for this persona/job combination and evaluates against those specifically rather than the full compliance universe. This reduces false-positive DENY verdicts for obligations not relevant to the current job.

Stage 5 — CAD Compilation (Enhanced)

The final Compiled AI Directive now includes: the Requirement Context Bundle (requirements governing this interaction), DesiredOutcome metrics as explicit success criteria, AcceptanceCriteria as response validation gates for the Service Facade, scenario-specific retrieval priority instructions for the domain vector DB query, and governance gap flags requiring human acknowledgment.

8. Requirements-Driven Vector Database Indexing

Domain vector databases are not generic semantic search indices. With requirements integration, they become structured knowledge stores organized by persona, job, and scenario — enabling precise, context-relevant retrieval that goes far beyond keyword or naive embedding similarity.

8.1 Indexing Taxonomy

Every document, scenario record, and knowledge artifact in the domain vector DB is indexed with a structured metadata envelope derived from the Requirements Ontology. This envelope governs retrieval — similarity search operates across both the semantic embedding and the metadata filters simultaneously.

Metadata Field	Source	Retrieval Role
persona_classes	req:forPersona → Persona class hierarchy	Filter: only retrieve content relevant to the querying persona's role and sub-roles
job_ids	req:describesJob → Job individual URIs	Boost: content indexed for the active job is retrieved at higher ranking
desired_outcome_ids	req:targetsOutcome → DesiredOutcome URIs	Boost: content directly relevant to active desired outcomes is priority-ranked
domain_id	req:inDomain → BusinessDomain URI	Hard filter: content outside the active domain is excluded unless cross-domain flag is set
sub_domain_ids	req:inSubDomain → BusinessSubDomain URIs	Soft filter with decay: sub-domain match boosts, non-match penalizes but does not exclude
scenario_context_ids	req:scenarioContext → ScenarioContext URIs	Exact match boost: scenario-specific content is ranked highest when context matches exactly
compliance_refs	req:governedBy → ComplianceObligation URIs	Compliance content tagged with active obligations is included regardless of similarity score

requirement_ids	Requirement individual URIs	Provenance: every retrieved chunk traces back to the requirement(s) that caused it to be indexed
acceptance_criteria_ids	req:hasCriterion → AcceptanceCriterion URIs	Validation: retrieved content is cross-referenced against acceptance criteria at agent evaluation time
nfr_quality_tags	NonFunctionalRequirement → qualityDimension values	Performance filtering: retrieval latency targets derived from active NFR performance thresholds
version_effective_from	req:version + effective date	Temporal filter: scenario details indexed under deprecated requirements are excluded from active queries

8.2 Scenario-Specific Detail Capture

ScenarioRequirement individuals are the primary vehicle for capturing the rich situational context that differentiates one use of a system from another. Each scenario is a named, versioned ontology individual with the following structure:

ScenarioRequirement Structure

Scenario ID + version | Activating Persona(s) | Triggering JobContext (conditions that make this scenario active) | Precondition Assertions (what must be true before this scenario applies) | Entity Scope (which ontology individuals are in scope) | Applicable Compliance Requirements (which regulations/standards govern this specific scenario) | Expected AI Behavior (what the AI must and must not do in this scenario) | AcceptanceCriteria (SPARQL-evaluable tests on the AI response) | Scenario-Specific Examples (few-shot exemplars stored as vector DB entries indexed exclusively to this scenario) | Escalation Conditions (when the scenario exceeds AI scope and requires human handoff)

8.3 Vector DB Query Modification by Requirements

When the Semantic Gateway dispatches a retrieval request to a domain agent's vector DB, the query is enriched with a Requirements-Derived Query Envelope (RDQE) that modifies every aspect of retrieval:

RDQE Element	Source in RCB	Effect on Retrieval
Job Embedding Bias	Active FunctionalJob embedding vector	Query embedding is a weighted blend of content embedding + job embedding — retrieval is biased toward job-relevant chunks
Outcome Keyword Boost	DesiredOutcome metric labels	Chunks containing outcome metric terminology receive score bonus
Scenario Pinning	Matching ScenarioContext IDs	Chunks tagged with active scenario IDs are retrieved first regardless of similarity rank
Compliance Mandatory Fetch	Active ComplianceRequirement obligation refs	Chunks tagged with mandatory compliance refs are retrieved unconditionally and prepended to context
Barrier Suppression	Active JobBarrier entities	Chunks known to contain barrier-causing terminology inconsistencies are tagged for disambiguation before delivery to agent

Persona Access Filter	Persona data access scope from Identity Ontology	Vector DB retrieval respects persona-scoped content access — private scenario data only retrieved for authorized personas
NFR Retrieval Budget	Active NonFunctionalRequirement performance thresholds	Maximum retrieval latency and chunk count governed by performance NFR — fast-path retrieval for time-critical scenarios

9. Requirements Coverage Across Domains and Sub-Domains

The following table illustrates how the requirements framework applies across key enterprise domains. Each domain has its own requirement instances, job hierarchies, SHACL shapes, SWRL rules, and vector DB partitions — all federated under the shared Requirements Ontology.

Domain	Sub-Domains	Requirement Types Active
Risk Management	Credit Risk, Operational Risk, Market Risk, Model Risk, Third-Party Risk	- JobReq: 'Assess residual risk for portfolio X under stress scenario Y' - ComplianceReq: ISO 31000 risk assessment coverage - ControlReq: Risk appetite threshold enforcement (SHACL sh:RiskAppetiteShape) - NFReq: Assessment completed within 4-hour SLA - ScenarioReq: Board reporting cycle — specific output format and completeness criteria
Data Governance	Data Quality, Privacy, Lineage, Master Data, Metadata Management	- JobReq: 'Certify data product for regulatory submission' - ComplianceReq: GDPR Art. 30 processing records, CCPA consumer rights - ControlReq: Data quality minimum thresholds per dimension (SHACL sh:DataQualityShape) - ProcessReq: Data stewardship review workflow before publication - ScenarioReq: Cross-border data transfer — jurisdiction-specific consent and transfer impact assessment
Security & Access	Identity, Privileged Access, Threat Intelligence, Incident Response, Vulnerability Management	- JobReq: 'Investigate anomalous access pattern for user U in system S' - ComplianceReq: NIST CSF Detect function, ISO 27001 Annex A.12 - ControlReq: MFA re-verification before privileged action (SWRL rule SEC-007) - NFReq: Threat correlation response under 30 seconds for P1 incidents - ScenarioReq: Zero-day advisory received — immediate asset exposure assessment
Compliance & Legal	Regulatory Reporting, Contract Management, Privacy Law, Audit Management, Litigation	- JobReq: 'Produce evidence package for regulatory examination request' - ComplianceReq: SOX Section 302/906 certification requirements, GDPR DPA obligations - ControlReq: Audit evidence completeness shape (sh:AuditEvidenceShape)

		- ProcessReq: Legal review gate before external regulatory submission - ScenarioReq: Regulatory inquiry with 30-day response window — priority evidence triage
AI Governance	Model Risk Management, Bias & Fairness, Explainability, EU AI Act Compliance, Model Lifecycle	- JobReq: 'Validate production model for continued use in high-risk decision context' - ComplianceReq: EU AI Act Art. 9 risk management, NIST AI RMF GOVERN function - ControlReq: Bias assessment within 90-day window (SHACL sh:AIModelGovernanceShape) - NFReq: Explainability report generated within 5 minutes of decision - ScenarioReq: Model drift detected — expedited revalidation with human-in-the-loop checkpoint
Analytics & Reporting	Financial Analytics, Operations Analytics, Customer Analytics, Regulatory Reporting	- JobReq: 'Produce board-ready risk dashboard for Q3 reporting cycle' - ComplianceReq: Statistical validity requirements, SOX disclosure requirements - ControlReq: Analytic output validation against methodology documentation (sh:AnalyticQualityShape) - NFReq: Dashboard refresh latency < 10 seconds for live operational metrics - ScenarioReq: Restatement scenario — re-run prior period analytics with audit trail

10. Acceptance Criteria as Automated Validation Gates

AcceptanceCriterion individuals are not documentation artifacts — they are executable tests. When an AI Agent returns a response, the Service Facade evaluates the response against all AcceptanceCriteria from the active Requirement Context Bundle before delivery. Non-passing responses are recycled with structured correction instructions.

10.1 Acceptance Criterion Types

Criterion Type	Evaluation Method	Example
Structural Completeness	JSON Schema or SHACL shape validation on response JSON	Risk assessment response must contain: risk_id, likelihood_score, impact_score, residual_risk, control_recommendations, ISO_31000_clause_reference
Ontology Consistency	SPARQL ASK query on response-derived triples	All entity references in response must resolve to valid individuals in the domain ontology — no undefined terms
Outcome Metric Coverage	Keyword/entity presence check mapped to DesiredOutcome metrics	Response must address all DesiredOutcome metrics active in the RCB — missing metric triggers specific clarification request
Compliance Citation	Reference validation against ComplianceRequirement governing_standards	Compliance analysis must cite applicable standard sections — assertions without citation are flagged as UNGROUNDED

Persona Scope Compliance	Cross-reference response entity list against persona access scope	No entity in response may exceed the querying persona's data access clearance level
NFR Quality Gate	Latency measurement, response length, readability score	Response delivered within performance NFR threshold; length proportional to complexity NFR specification
Scenario-Specific Test	Custom SPARQL or Python evaluation against scenario fixture	For board reporting scenario: response format matches board template schema, confidence intervals included, no raw data exposed

11. Requirements Governance Operations

11.1 Requirements Lifecycle Management

- **Versioning:** Requirements follow semantic versioning aligned to ontology releases. Breaking changes to acceptance criteria require a major version increment and reactivation workflow.
- **Deprecation:** Deprecated requirements are retained in the triplestore with status=Deprecated but excluded from gateway queries. Vector DB entries are marked inactive but preserved for audit.
- **Conflict Resolution:** The req:conflictsWith property is bidirectional. When triggered by SWRL rule REQ-008, a structured conflict record is generated in the governance queue for domain steward resolution within a defined SLA.
- **Coverage Dashboard:** A requirements coverage metric tracks: % of active Personas with at least one JobRequirement, % of ComplianceRequirements with ControlRequirement linkage, % of ControlRequirements with enforcedBy SHACL/SWRL linkage, % of ScenarioRequirements with SPARQL-evaluable acceptance criteria.
- **Requirements Traceability Matrix:** Generated on-demand from the triplestore via SPARQL: maps every active AI behavior (CAD element, PEP verdict, agent capability) back to the requirement(s) that mandate it. Surfaces gaps where AI behaviors have no requirement basis.

11.2 Feedback Loop: Requirements Evolution

The architecture supports closed-loop requirements evolution. When an agent response fails an AcceptanceCriterion, the failure is logged as a RequirementsFinding individual linked to the failing criterion. When a sufficient number of findings accumulate on a criterion, a RequirementsReview is automatically triggered, prompting domain stewards to either adjust the criterion or improve the AI capability that failed it. This ensures requirements remain calibrated to actual system performance.

- **Finding → Review Threshold:** Configurable per requirement priority: Must-priority criteria trigger review after 3 failures within 30 days; Should-priority after 10 failures.
 - **Prompt Template Feedback:** Systematic CAD compilation failures on specific job/persona combinations trigger prompt template review, linking poor AI behavior back to the engineering pattern, not just the model.
 - **Ontology Gap Detection:** Terms in user prompts that fail ontology resolution are logged as OntologyGap individuals, aggregated weekly, and prioritized for ontology extension based on frequency × persona priority.
-

12. Integration Summary: What Changes in Each Architectural Zone

Zone	What the Requirements Integration Adds
Zone 1: Prompt Engineering	SPO gains req_context_query field; elicitation UI surfaces JTBD questions; persona registry links to JobRequirement instances; prompt templates versioned against requirement versions
Zone 2: Semantic Gateway	Stage 0 requirement resolution added; CAD gains RCB attachment, DesiredOutcome metrics, AcceptanceCriteria gates, governance gap flags; SWRL REQ-rules activate
Zone 3: Ontology & Rules Store	Requirements Ontology module added to federation; sh:Requirement* shapes in relational SHACL store; REQ-SWRL rules in relational rules store; Job and DesiredOutcome ontology modules deployed
Zone 4: Service Facade	Context-Content Mapper uses RCB for asset resolution priority; AcceptanceCriteria evaluated on all agent responses before delivery; scenario-specific retrieval priority applied to vector DB queries
Zone 5: AI Agents + Vector DBs	All vector DB entries carry requirements metadata envelope; RDQE modifies every query with job, outcome, scenario, and persona filters; scenario-pinned entries are retrieved before general domain results
Zone 6: Legacy Integration	Downstream calls are annotated with requirement IDs for end-to-end traceability; NFR performance thresholds govern integration SLAs; compliance requirement citations are attached to data retrieval audit records